

A3 - Distributed Data Processing with Apache Spark

Arnab Kumar Ghosh

March 3, 2026

1 Part 1: Apache Spark Fundamentals

1.1 Q1.1

A **transformation** (e.g. `map`, `filter`) creates a new RDD or DataFrame but does not run anything. Spark just saves it in a DAG. An **action** (e.g. `count`, `collect`) actually runs the DAG on the cluster and gives back a result.

Transformation Example (no immediate computation):

```
rdd = spark.sparkContext.parallelize(range(100))
```

No data is processed here. Spark only records the step in the DAG.

Action Example (triggers computation):

```
rdd.count() # returns 100
```

`count()` runs the DAG on the cluster and returns 100 to the driver.

Spark uses lazy evaluation so it can look at the full plan first, combine steps, and reduce unnecessary work before touching any data.

1.2 Q2.1

Lazy evaluation means Spark does not run anything when a transformation is called. It just saves the step in a DAG. Computation only starts when an action is called, at which point Spark runs the full DAG on the cluster.

Transformation Example (no immediate computation):

```
en_tokens = preprocess(en_rdd) # map -> map -> map chain
```

This chains three `map` steps (lowercase, split, remove empty tokens). No data is read or processed yet – Spark only adds these steps to the DAG.

Action Example (triggers computation):

```
en_tokens.take(10)
```

`take(10)` is an action. It runs the DAG: reads from HDFS, applies the `map` steps, and returns the first 10 lines to the driver. Likewise, `en_rdd.count()` is an action that counts all lines in the file.

1.3 Q2.2

1. **Intermediate Data – Memory vs. Disk:** MapReduce saves intermediate data to disk after every Map and Reduce step. Spark keeps it in memory and only writes to disk when memory is full. This makes Spark much faster for multi-step jobs.

2. **Multi-Step Computation – DAG vs. Fixed Map-Reduce:** MapReduce only supports a fixed Map then Reduce structure. For complex jobs, we need to chain many separate MapReduce jobs, each reading and writing HDFS. Spark uses a single DAG that can hold any number of steps and runs them all in one job.

1.4 Q3.1

There are three reasons to define the schema manually instead of using `inferSchema=True`.

1. **It is faster.** With `inferSchema`, Spark reads the whole file twice – once to guess the column types, and once to actually load the data. The parking dataset is about 1.6 GB, so Spark reads 1.6 GB twice. If we define the schema ourselves, Spark only reads the file once.
2. **It gives correct types.** Spark’s guessing is sometimes wrong. For example, the column `Plate Expiry Date` contains values like 202312. Spark guesses it is an integer, but it should be a string. The column `Fine amount` can also be misread if some rows are empty or have mixed formats. When we define the schema ourselves, we control the type of every column.
3. **It is safer in production.** In a real pipeline, new data files arrive every day. If one file has a slightly different format, `inferSchema` may silently assign the wrong type and the rest of the pipeline produces wrong results without any warning. With an explicit schema, Spark immediately throws an error if the data does not match. We find the problem right away instead of later.

1.5 Q3.2

1.5.1 (a) One optimisation between the Analyzed and Optimized Logical Plans

Column pruning. The query only needs the `Fine amount` column. In the Analyzed Plan, Spark still keeps all 22 columns in the plan. In the Optimized Plan, Spark removes the 21 unused columns early:

```
+ - Project [Fine amount#16]
+ - Relation [...all 22 cols...] csv
```

This means Spark reads only one column from the CSV file instead of all 22. This saves both reading time and memory.

1.5.2 (b) UDF vs. Native when/otherwise Comparison

Table 1: UDF vs. native Spark SQL – comparison based on `explain(True)` output.

Aspect	Python UDF	Native when/otherwise
Optimized-plan operator	BatchEvalPython	Inline CASE WHEN
Visible to Catalyst?	No	Yes
Constant folding applied?	No	Yes (<code>cast</code> → literal)
Data serialisation overhead?	High (JVM ↔ Python)	None
Predicate pushdown possible?	No	Yes
Recommended for production?	No	Yes

The `explain(True)` output shows this clearly. The Python UDF plan has a `BatchEvalPython` node. This means Spark sends each row from the JVM to a Python process, waits for the result, and gets it back. This back-and-forth is slow. The native SQL plan has no such node. The `CASE WHEN` expression runs directly inside Spark on the JVM. Spark can also see inside the expression and optimise it, which it cannot do with a Python function.

2 Part 2: Advanced Data Analytics – Structured Streaming

2.1 Q4.1

What the watermark does:

The watermark tells Spark how long to wait for late data before closing a time window. Spark tracks the latest event time it has seen and subtracts the watermark duration to get a cutoff time. Any window older than this cutoff is closed and removed from memory.

In this task I used a **30-minute window** and a **15-minute watermark**.

30-minute-late tweet with a 15-minute watermark:

Suppose the latest event time seen by Spark is 11:00. The cutoff is $11:00 - 15 \text{ min} = 10:45$. A tweet with event time 10:30 is before the cutoff, so Spark will **drop it**. Its 30-minute window is already closed and any state for that window is gone.

No watermark:

Without a watermark, Spark never closes any window. All late data is kept forever. The state stored in memory keeps growing and the stream will eventually run out of memory.

2.2 Q4.2

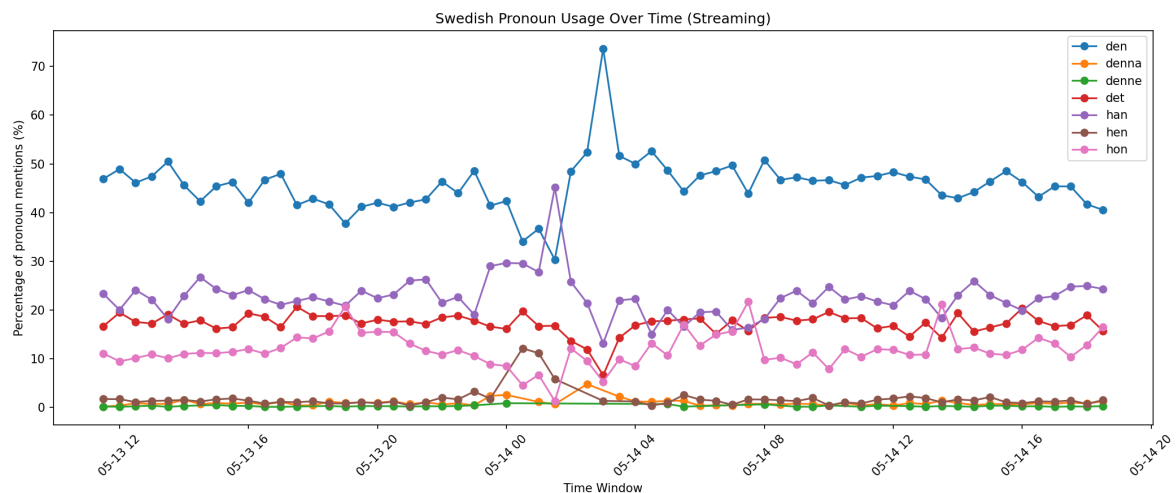


Figure 1: Swedish pronoun usage percentage over 30-minute windows (streaming, May 13–14 2014).

Figure 1 shows the pronoun *den* clearly dominates throughout the whole period, making up roughly 40–50% of all pronoun mentions in most windows. There is one sharp spike around midnight on May 14 where *den* jumps to about 74%, which likely reflects a burst of tweets about a specific topic or event at that time. The gender-neutral pronoun *hen* stays very low (near 0–2%) across all windows, showing it was still rarely used in Swedish Twitter in 2014.

2.3 Q4.3

Tasks 1–3 used the classic **batch** model: all data is loaded at once and results are returned after the full job finishes. Task 4 used **Structured Streaming**: data arrives continuously, and Spark processes it in small micro-batches, updating results in near real-time. Table 2 compares the two paradigms across five key dimensions.

Table 2: Spark Batch (Tasks 1–3) vs. Structured Streaming (Task 4).

Dimension	Spark Batch (Tasks 1–3)	Spark Structured Streaming (Task 4)
Latency (when are results available?)	Only after the full dataset is processed. <code>sc.textFile()</code> (Task 2) and <code>spark.read.csv()</code> (Task 3) load everything first. Each action like <code>show()</code> or <code>takeOrdered()</code> blocks until the whole RDD/DataFrame is scanned.	After each micro-batch. <code>spark.readStream.json()</code> picks up new tweet files as they land in HDFS and pronoun counts update within seconds.
Development complexity	Lower: standard RDD (<code>map</code> , <code>reduceByKey</code> , <code>join</code>) and DataFrame (<code>groupBy</code> , window functions, UDFs) operations. No state or timing concerns.	Higher: requires setting up a schema, watermark, tumbling window, output mode, checkpoint location etc.
Fault tolerance mechanism	RDD lineage: any lost partition is recomputed from the original source files on HDFS using the DAG.	Checkpointing + write-ahead logs: <code>StateStoreSave</code> writes window state to HDFS after each micro-batch, enabling exactly-once recovery.
Handles late data?	Not applicable: all data is present before processing starts, so there is no concept of late arrival.	Yes: the 15-minute watermark defines a grace period. A tweet arriving up to 15 min late is still counted. Anything older is dropped and its window state freed.
Typical use case	Offline analytics and ETL: word frequency over Europarl corpus (Task 2), parking violation analysis and query plan comparison (Task 3).	Near real-time monitoring: tracking Swedish pronoun usage trends as tweets stream in (Task 4).